

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 06 -

PIPELINES INTEGRADOS DVC + MLFLOW

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	19
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS	24

EMSE

O QUE VEM POR AÍ?

Nesta aula, o foco está na integração de pipelines de dados e modelos empregando DVC e MLflow, consolidando uma abordagem completa de Machine Learning Engineering orientada a MLOps. Em aulas anteriores, você aprendeu a versionar dados com o DVC, a armazená-los remotamente integrado ao GitHub, a rastrear experimentos com o MLflow e a gerenciar o ciclo de vida de modelos em um Model Registry. Agora, vamos orquestrar todos esses elementos em pipelines integrados, nos quais dados, código, experimentos e modelos evoluem de forma sincronizada, automatizada e reproduzível.

Ao longo desta aula, você entenderá como estruturar pipelines de Machine Learning que atendam às exigências de escala, governança, confiabilidade e colaboração – pilares das práticas modernas de MLOps. Prepare-se para descobrir como conectar todas as etapas do fluxo de ML (da ingestão de dados ao deploy de modelos) em um mecanismo coeso, reduzindo a intervenção manual e os riscos de inconsistência. Esta introdução “quebra o gelo” sobre pipelines integrados, instigando você a pensar em como montar seu próprio pipeline de ML de ponta a ponta.

HANDS ON

A fase prática desta aula simula um pipeline completo de Machine Learning, semelhante aos empregados em ambientes corporativos. Durante o Hands On, você irá estruturar o pipeline, definindo as etapas de ingestão de dados, pré-processamento, treinamento e avaliação, com scripts organizados e entradas/saídas padronizadas para cada etapa. Em seguida, criará um pipeline com DVC, declarando dependências entre etapas e versionando automaticamente dados intermediários. O DVC permitirá executar o pipeline de forma incremental, reexecutando apenas as etapas impactadas por mudanças no código, nos dados ou nos parâmetros.

Paralelamente, utilizaremos o MLflow para integrar o rastreamento de experimentos ao pipeline: a cada execução de treinamento, parâmetros, métricas e artefatos (como o modelo treinado) serão logados automaticamente, associando cada run a versões específicas dos dados e do código, garantindo rastreamento completo do experimento. Por fim, veremos a integração com Git, colocando todo o código e metadados (arquivos de configuração DVC, parâmetros etc.) sob versionamento. Assim, cada commit no repositório representará um estado completo e reproduzível do projeto, unificando DVC e MLflow ao controle de versão tradicional.

Código completo disponível no GitHub: [kiran-91/Data-Pipeline-with-DVC-and-MLflow-for-Machine-Learning](https://github.com/kiran-91/Data-Pipeline-with-DVC-and-MLflow-for-Machine-Learning) (repositório demonstrando um pipeline integrado de ML).

Assista às videoaulas para ver a execução completa desse pipeline integrado na prática, com orientação passo a passo sobre cada comando e análise detalhada dos resultados obtidos.

SAIBA MAIS

Projetos de Machine Learning que não contam com pipelines integrados enfrentam desafios significativos. Em ambientes manuais e fragmentados, ocorrem execuções inconsistentes, falta de rastreabilidade de dados e modelos, dificuldade de reprodução de resultados e alto risco de erro humano, além de baixa escalabilidade operacional conforme os experimentos crescem em complexidade. Tais limitações acarretam custo extra e dificultam a manutenção dos sistemas de ML. Por exemplo: ajustes ou debugs tornam-se tarefas árduas sem garantias de reproduzir o cenário original. Sculley et al. (2015) observaram que em um sistema de ML real, o código do modelo em si é apenas uma pequena fração do total – há uma enorme quantidade de “código de colagem” (glue code) e componentes auxiliares (coleta e validação de dados, configuração, análise de modelos, monitoramento etc.) que acumulam dívida técnica quando gerenciados de forma ad-hoc. Em outras palavras, é perigoso obter pequenos insights rápidos com ML sem pensar no longo prazo: sem uma arquitetura adequada, os ganhos iniciais se perdem em sistemas complexos e irregulares.

Pipelines integrados emergem como solução para esses problemas, pois introduzem automação, padronização e reprodutibilidade no ciclo de vida de ML. Em vez de execuções manuais e sujeitas a lapsos, o pipeline automatiza o fluxo de trabalho inteiro, garantindo que cada experimento siga os mesmos passos definidos. Isso reduz retrabalho e erro humano (por exemplo, evita rodar etapas fora de ordem ou com dados errados) e aumenta a confiabilidade operacional. Segundo um estudo, a ausência de versionamento de dados/modelos e de rastreamento de experimentos em pipelines manuais gera falta de reprodutibilidade e dificuldades futuras na manutenção – problemas que um bom pipeline mitiga ao encapsular cada etapa com suas dependências e resultados versionados.

Em termos de produtividade, pipelines integrados permitem iterações rápidas com segurança: sabe-se exatamente o que mudou de um experimento para outro e quais partes precisam ser reprocessadas. Isso agiliza a inovação em modelos, pois os times podem tentar novas ideias sem medo de “quebrar” algo existente – qualquer resultado pode ser restaurado a partir do histórico versionado. Não por acaso, integrar pipelines de ML ao estilo DevOps é hoje considerado uma prática fundamental de

MLOps (Machine Learning Operations) para levar modelos de forma consistente à produção. Em suma, pipelines bem projetados resolvem os “pitfalls” (armadilhas) de se operar sistemas de ML complexos: automatizam o que é repetitivo, conferem rastreabilidade total dos dados e modelos, facilitam reproduzir ou desfazer experimentos e escalam o desenvolvimento de ML dentro das organizações. Grandes empresas de tecnologia até desenvolveram suas próprias plataformas internas orientadas a pipeline (e.g. Uber Michelangelo, Facebook FBLearner Flow), comprovando ganhos significativos em agilizar e padronizar o fluxo de ML.

Insights: de acordo com relatos recentes, companhias que adotaram pipelines robustos de MLOps vêm colhendo benefícios concretos: ciclos de deployment ~40% mais rápidos e redução de ~60% em incidentes de deriva de modelo após implementar DVC e MLflow em produção. Isso reforça que investir em automação e versionamento integral do workflow de ML não é apenas uma questão técnica, mas também econômica – diminuindo tempo de inatividade, esforço de engenharia e riscos de erros custosos.

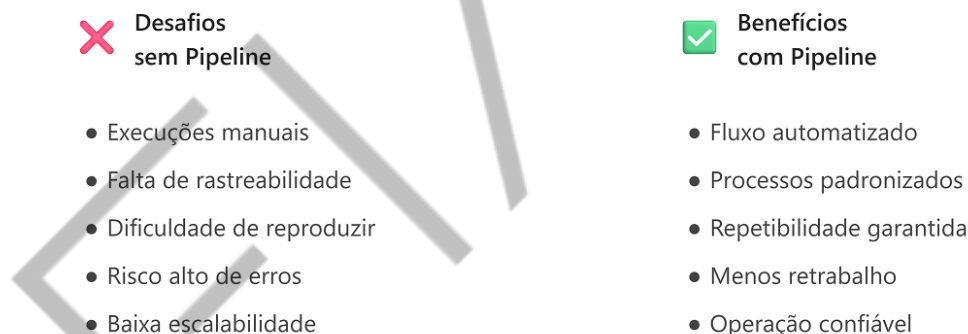


Figura 1 – MLOps
Fonte: Elaborado pelo autor (2026)

Sem pipelines integrados, projetos de ML sofrem com inconsistência e difícil manutenção; já pipelines bem planejados trazem automação, padrão e confiabilidade.

DVC: versionamento de dados e orquestração de pipelines

O DVC (Data Version Control) atua como orquestrador de dados e dependências no pipeline. Ele foi concebido justamente para levar ao mundo de Data Science as boas práticas de engenharia de software, como o versionamento e rastreamento completo de artefatos. Em essência, o DVC oferece ao indivíduo cientista de dados um “Git para dados e modelos”: é uma ferramenta de linha de comando open-source que versiona datasets, modelos e até definições de pipelines, tornando experimentos reproduzíveis e compartilháveis entre equipes. No contexto de pipelines integrados, o DVC desempenha vários papéis fundamentais. Em primeiro lugar, todo conjunto de dados (grande ou pequeno) pode ser armazenado de forma versionada. O DVC substitui arquivos grandes no repositório por pequenos metadados (ponteiros) que apontam para o conteúdo real em um armazenamento externo (um remote como S3, Google Drive, Azure Blob etc.). Dessa forma, é possível recuperar qualquer versão anterior de um dataset conforme necessário, garantindo experimentos comparáveis ao longo do tempo.

Além disso, nas etapas do pipeline, o DVC rastreia também arquivos gerados (por exemplo, dados pré-processados, modelos treinados). Cada arquivo ou diretório monitorado recebe um hash único (calculado via MD5, por exemplo) para identificar seu conteúdo. Se o conteúdo não muda, o DVC reutiliza o resultado em vez de recriá-lo – isso constitui o mecanismo de cache inteligente do DVC, que evita duplicação de arquivos e repetições desnecessárias de computação. No cache local do DVC, cada versão única de um arquivo fica armazenada somente uma vez, economizando espaço e permitindo reverter a qualquer momento.

Através do arquivo `dvc.yaml` (como vimos no Hands On), o DVC permite declarar cada etapa (stage) do pipeline com seus comandos, entradas (dependências) e saídas. Assim, ele constrói internamente um grafo de dependências do pipeline. Quando executamos `dvc repro`, o DVC compara os hashes dos inputs de cada etapa com o que foi registrado na última execução (no arquivo `dvc.lock`). Se nada mudou, aquela etapa é ignorada; mas se algum input foi alterado, a etapa é executada novamente e suas saídas são atualizadas. Esse recurso de execução incremental acelera drasticamente o ciclo de desenvolvimento em projetos grandes – por exemplo,

se apenas o hiperparâmetro do modelo mudou, não há por que reprocessar os dados brutos desde o início e o DVC garante que apenas o necessário seja recomputado.

E ao versionar tudo que entra e sai do pipeline, o DVC viabiliza a reprodução exata de qualquer experimento. Usando Git para versionar o código e os metadados do DVC (como o `dvc.lock` e `params.yaml`), conseguimos associar cada commit do código a versões específicas dos dados e modelos. Isso significa que, dado um commit no Git, o DVC pode recuperar as versões corretas dos datasets de entrada e dos modelos gerados, permitindo rodar todo o pipeline naquele estado e obter os mesmos resultados de antes. Essa propriedade é essencial em cenários de auditoria, comparações ou rollbacks (voltar a um modelo anterior caso um modelo novo apresente problemas).

Em termos de implementação, o DVC integra-se ao Git de maneira elegante. Arquivos de dados versionados pelo DVC ganham um arquivo de acompanhamento com extensão `.dvc` (que contém o hash e localização do conteúdo real). No repositório Git, versionamos esses arquivos `.dvc` em vez do dado cru – assim, o Git versiona metafiles pequenos (facilitando fusões e histórico), enquanto os dados volumosos ficam gerenciados pelo DVC externamente. No momento de clonar o projeto, basta executar `dvc pull` para baixar os dados necessários do repositório remoto configurado. Esse design alivia o Git de armazenar gigabytes de dados, mas mantém o histórico único e sincronizado de código + dados: cada commit traz embutido quais versões de dados usar.

Formalismo matemático (hashing e reexecução)

Podemos representar o mecanismo de decisão do DVC de forma simplificada. Suponha uma etapa do pipeline que depende de um conjunto de arquivos de entrada $D = \{d_1, d_2, \dots, d_n\}$ e produz um arquivo de saída O . O DVC calcula um identificador de conteúdo para cada arquivo, como por exemplo uma função hash $h(\cdot)$ (MD5 ou SHA-256). No estado anterior registrado, havia um conjunto de hashes $H_{\text{prev}} = \{h(d_1)_{\text{prev}}, \dots, h(d_n)_{\text{prev}}\}$ e um hash para a saída $h(O)_{\text{prev}}$. Ao rodar `dvc repro`, o DVC recalcula $H_{\text{curr}} = \{h(d_1)_{\text{curr}}, \dots, h(d_n)_{\text{curr}}\}$. Se $H_{\text{curr}} =$

H_{prev} (nenhuma mudança nos inputs), então ele assume que O permanece válido e reutiliza a versão em cache daquele O_{prev} . Caso contrário ($H_{\text{curr}} \neq H_{\text{prev}}$), a etapa é reexecutada para gerar um novo output, cujo hash O_{curr} será então armazenado no novo `dvc.lock`.

Esse procedimento garante correção e eficiência: O só é recalculado se alguma entrada mudou, caso contrário, a versão anterior de O é determinística e permanece válida. Trata-se de uma forma de memoização robusta aplicada a pipelines de dados.

Reprodutibilidade ponta a ponta e conexão com MLOps

Com a integração total de DVC (dados/pipeline) + MLflow (experimentos/modelos) + Git (código), atingimos reprodutibilidade ponta a ponta. Isso significa que podemos selecionar qualquer experimento passado e recriá-lo exatamente. Todos os elementos necessários estão versionados: o código do pipeline (scripts etc.) está no Git; os dados do experimento (brutos e processados) estão no DVC; os parâmetros e configurações estão no `params.yaml` versionado; e os resultados (métricas e modelo final) estão logados no MLflow e/ou armazenados pelo DVC. Essa riqueza de rastros permite não apenas reproduzir resultados para fins de auditoria ou pesquisa, mas também realimentar o processo de desenvolvimento. Por exemplo, se um novo membro do time quiser entender o porquê de certa decisão, pode examinar os experimentos passados e ver, digamos, que um determinado ajuste de hiperparâmetro não melhorou a métrica em março de 2024 (porque há um run registrado mostrando isso).

Essa reprodutibilidade total é um dos objetivos-chave do movimento de MLOps, que aplica princípios de DevOps à Machine Learning. No contexto mais amplo de MLOps, pipelines integrados são a espinha dorsal que conecta o desenvolvimento de modelos à operação contínua em produção. Em um cenário ideal de MLOps, ao se integrar tudo que discutimos neste pipeline a sistemas de CI/CD, se obtém um fluxo onde novos dados ou código disparam automaticamente novas execuções de pipeline (Continuous Training) e, se aprovadas, alimentam diretamente a atualização do

modelo em produção (Continuous Delivery). Ou seja, o pipeline integrado torna-se parte do “motor” de modelos em produção, rodando periodicamente ou sob demanda para manter o modelo atualizado e monitorado.

No nosso escopo, focamos na porção offline (desenvolvimento e experimentação), mas é importante notar que a mesma estrutura de pipeline integrada facilita muito a transição para produção. Por exemplo, uma vez que temos scripts claros para cada etapa e tracking completo, podemos colocá-los em um agendador de tarefas (como Airflow, Kubeflow Pipelines, Azure ML Pipeline etc.) para rodar regularmente com dados novos. Os artefatos versionados pelo DVC/MLflow podem ser promovidos para ambientes de produção com confiança, pois sabemos exatamente de qual experimento vieram e podemos auditá-los a qualquer momento.

Arquitetura de pipelines modernos (princípios de projeto)

Ao construir pipelines integrados, adotamos princípios de engenharia de software para garantir modularidade, testabilidade e manutenibilidade. Para começar, cada etapa do pipeline deve ter uma função bem definida e única. Por exemplo, pré-processamento trata apenas de transformar os dados brutos em uma forma utilizável; treinamento foca em treinar o modelo; avaliação apenas calcula métricas de performance; deploy cuida da implantação do modelo em produção. Essa separação clara permite desenvolvimento paralelo (diferentes pessoas podem trabalhar em etapas distintas sem conflitos) e facilita testes independentes de cada componente. Além disso, mudanças em uma etapa (por exemplo, ajustar como os dados são normalizados no pré-processamento) não devem quebrar etapas subsequentes, desde que a interface (formato dos dados de saída) permaneça a mesma – isso segue o princípio de baixo acoplamento. Em nosso pipeline, essa separação é evidente nos scripts distintos (`preprocess.py`, `train.py`, `evaluate.py`), orquestrados pelo DVC.

Pipelines modernos geralmente separam a forma declarativa da dependência entre etapas da forma imperativa de executar as tarefas. Em outras palavras, descrevemos o que fazer e as relações (por exemplo: “Para treinar, preciso do arquivo X gerado na etapa de preparação”) em um arquivo de configuração e deixamos o orquestrador decidir quando e como executar (por exemplo: “Se o arquivo X já estiver

atualizado, pule preparação; senão, execute na ordem certa”). No nosso caso, o `dvc.yaml` declara as etapas e dependências, sem especificar manualmente a ordem – o DVC analisa o grafo e determina a ordem ótima (incluindo possivelmente execução paralela de etapas independentes). Essa separação permite muita flexibilidade: podemos rodar `dvc repro` localmente ou em um servidor de CI/CD e ele sempre calculará a execução correta; podemos também paralelizar automaticamente etapas que não dependem uma da outra (DVC suporta `dvc repro -j N` para rodar `N` jobs em paralelo). Em pipelines mais complexos (por exemplo, com dezenas de etapas), essa análise automática de dependências evita erros de ordem de execução e aproveita ao máximo os recursos, rodando em paralelo o que for possível sem intervenção humana.

Conforme discutido, uma característica essencial é o uso de cache de resultados. Ferramentas como DVC implementam isso via hashes de conteúdo – se nada mudou, a saída não é recalculada. Esse padrão acelera exponencialmente o ciclo de desenvolvimento em projetos grandes, pois evita trabalho repetido. Em pipelines modernos, essa ideia pode se estender também a datasets incrementais: por exemplo, se o pipeline processa diariamente dados novos, idealmente queremos processar apenas as novas entradas em vez de toda a base diariamente. Embora o DVC não implemente diretamente a lógica de streaming ou delta incremental, podemos arquitetar nosso pipeline para tratar os dados em partições (ex.: por data), e a lógica de dependência do DVC então cuidará de rodar apenas as partições afetadas. Em suma, evitar recomputação desnecessária é um princípio-chave – caches determinísticos e particionamento de dados são técnicas para isso.

Todo output importante do pipeline deve ser versionado (via DVC ou equivalente). Isso permite rollbacks cirúrgicos – por exemplo, se uma nova versão do modelo em produção apresentar problemas, conseguimos rapidamente “voltar” ao modelo anterior porque o histórico do DVC/MLflow o preserva. Em nosso pipeline, qualquer versão passada do modelo pode ser recuperada pelo seu run ID no MLflow ou pelo commit Git associado. Em pipelines sem versionamento de outputs, ocorreria o risco de “treinar sobre” o modelo anterior e perdê-lo para sempre. Além disso, ao versionar outputs, conseguimos comparar entre versões facilmente (como com `dvc diff` ou comparando métricas no MLflow). Essa cultura de versionamento é emprestada

do Git, que transformou desenvolvimento de software com controle de versão de código, aplicada a todos os artefatos de ML.

Arquiteturas modulares de pipeline facilitam testes automáticos. Podemos escrever testes unitários para cada script: e.g., testar se `preprocess.py` está lidando corretamente com valores nulos nos dados, ou se `train.py` retorna um modelo com certas propriedades. Também é possível simular pequenos subconjuntos de dados para teste rápido local. Ao integrar com CI, podemos executar um pipeline em miniatura a cada commit. Por exemplo, treinando um modelo em uma amostra reduzida só para verificar se a pipeline roda sem erros. Essa segurança de testes é crucial conforme pipelines vão para produção, para evitar quebrar fluxos com atualizações – falaremos adiante sobre estratégias de testes em pipelines.

Em linhas gerais, projetar pipelines de ML segue os mesmos princípios de engenhar software confiável. Pipeline de ML não é “código descartável de pesquisa”, mas sim a fábrica que produz modelos continuamente – logo, aplicar engenharia rigorosa nele traz enorme retorno. Como nota histórica, muitas dessas ideias foram cristalizadas após o famoso artigo “Technical Debt in ML Systems” de 2015, que expôs como pipelines ad-hoc geravam complexidade incontrolável. Hoje, ferramentas como DVC, MLflow, Kubeflow, Airflow e outras existem precisamente para endereçar esses desafios arquiteturais.

Padrões de integração dvc–mlflow no pipeline

Ao implementar DVC e MLflow em conjunto, alguns padrões e boas práticas ajudam a extrair o máximo benefício de cada ferramenta. O MLflow possui um recurso de autologging que captura automaticamente parâmetros e métricas de bibliotecas populares (scikit-learn, Keras, PyTorch etc.) sem necessidade de chamadas manuais de `mlflow.log_param` ou `mlflow.log_metric`. Habilitar o autolog no início do script (`mlflow.sklearn.autolog()`, por exemplo) pode eliminar código boilerplate e garantir que nada seja esquecido de logar. No contexto do pipeline, pode-se ligar o autolog e deixar o MLflow registrar hiperparâmetros do modelo e métricas como acurácia automaticamente quando `model.fit` e `model.score` forem chamados. Integrado com DVC, isso significa que toda vez que rodamos `dvc repro`, se a etapa de treinamento

reexecutar, o MLflow autolog já captura tudo sem esforço manual. Esse padrão reduz a chance de erro humano (ex.: esquecer de logar alguma métrica importante) e simplifica o código de treinamento, focando apenas na lógica algorítmica.

O uso de um arquivo de parâmetros (como `params.yaml` no DVC) centraliza as configurações do pipeline. O DVC integra isso muito bem – declaramos no `dvc.yaml` quais campos do `params.yaml` cada etapa utiliza e ele inclui esses valores no cálculo de hash de dependências. Assim, alterar um hiperparâmetro no `params.yaml` faz o DVC saber que a etapa de treinamento deve rodar de novo. Enquanto isso, no MLflow, podemos logar automaticamente todos os parâmetros lendo o próprio `params.yaml`.

Em nosso Hands On, fizemos `mlflow.log_params(params)` após carregar do YAML. Esse padrão de fonte única da verdade para parâmetros garante consistência: o valor de `learning_rate` usado de fato no código é o mesmo registrado no MLflow e o mesmo versionado pelo Git (via alteração no arquivo). Equipes maiores podem assim revisar mudanças de hiperparâmetros via Pull Requests do Git, por exemplo, tendo rastreabilidade de “quem mudou tal parâmetro e quando”. Em pipelines complexos, é recomendável segmentar o `params.yaml` em seções (ex.: `preprocess`, `train` etc.) para organizar múltiplos parâmetros. Vale mencionar que o MLflow autolog muitas vezes captura esses parâmetros automaticamente quando modelos são instanciados – por exemplo, ao criar `RandomForestClassifier(n_estimators=100)`, o autolog do MLflow registra `n_estimators=100`. Mas usar o YAML como intermediário é útil quando vários scripts distintos precisam ler parâmetros em comum.

Como mencionado, podemos aproveitar tanto o DVC quanto o MLflow para métricas. Uma prática é configurar a etapa de avaliação para gerar um arquivo JSON de métricas (contendo, por exemplo, `{"accuracy": 0.918, "f1": 0.882}`) e marcá-lo no `dvc.yaml` como `metrics:`. Assim, o DVC armazena essas métricas e consegue fazer `dvc metrics show` ou `dvc metrics diff` entre commits para ver se melhoraram ou pioraram. Simultaneamente, dentro do código `evaluate.py`, usamos `mlflow.log_metric("accuracy", acc)` etc. para registrar no MLflow. Embora possa parecer duplicação, isso traz benefícios: o DVC permite a análise rápida via CLI e integra com Git (ex.: ao fazer checkout de um commit antigo, `dvc metrics show` mostra as métricas daquele commit sem precisar rodar nada), enquanto o MLflow oferece interface rica e histórico completo de runs. Outro padrão é salvar gráficos ou artefatos

(por exemplo, um plot de curvas de aprendizado) e usar tanto `dvc add` nesses gráficos quanto `mlflow.log_artifact`. Assim, os gráficos ficam versionados no storage do DVC (podendo ser recuperados por `commit`) e também facilmente visualizáveis no MLflow UI.

Uma integração poderosa é anotar dentro do MLflow o identificador da versão de dados utilizada. Como o DVC versiona dados via hash, podemos, por exemplo, armazenar no MLflow um tag chamada `data_version` com o hash (ou um ID amigável) do dataset de treino usado naquele run. O DVC fornece via API Python funções para obter o hash ou caminho de um determinado arquivo sob controle dele. No snippet do Hands On, poderíamos fazer: `data_hash = dvc.api.get_url('data/processed/train.csv')` (essa função, quando apontada para um arquivo sob DVC, retorna a URL no storage remoto daquela versão específica, que inclui o hash).

Esse `data_hash` pode então ser logado: `mlflow.set_tag("train_data_version", data_hash)`. O resultado é que, ao inspecionar um experimento no MLflow, saberemos exatamente com qual versão dos dados ele foi treinado. No caso de alguma divergência ou problema, podemos rastrear se um modelo foi treinado com dados desatualizados, por exemplo. Esse tipo de rastreamento cruzado dados-modelo é o sonho de um auditor de MLOps! Em eventuais auditorias ou verificações de compliance, consegue-se responder perguntas do tipo: “Modelo X (id=abc) foi treinado com quais dados? Esses dados estavam aprovados e completos até qual data?” – tudo porque tags e metadados foram registrados.

Em síntese, adotar padrões de integração como os descritos multiplica os ganhos do pipeline. DVC e MLflow deixam de ser ferramentas isoladas e passam a operar como um ecossistema coeso, cobrindo toda a linhagem de dados e modelos (data & model lineage). Essa sinergia aumenta a confiabilidade (cada mudança relevante é capturada por alguma ferramenta) e fornece informações ricas para gerenciamento; por exemplo, facilitando apresentações de resultados, pois podemos extrair gráficos e tabelas diretamente dos logs dos experimentos.

Otimização de performance de pipelines

Conforme os projetos de ML crescem (mais dados, modelos mais complexos), os pipelines podem ficar lentos se não forem bem projetados. Se o pipeline contém etapas independentes, devemos executar em paralelo sempre que possível para reduzir o tempo total. O DVC, por exemplo, pode rodar stages simultaneamente com o comando `dvc repro -j <n>`, onde `<n>` é o número de jobs.

Em um fluxo típico, enquanto a etapa de preparação de dados roda, talvez a de treinamento tenha que esperar (pois depende dos dados), mas se tivéssemos duas tarefas de treinamento com modelos diferentes que usam o mesmo pré-processamento, poderíamos paralelizá-las. Fora do DVC, orquestradores como Apache Airflow ou Luigi permitem definir pools de recursos e dependências para paralelizar tarefas. O importante é identificar tarefas que não têm dependência entre si e executá-las ao mesmo tempo.

No nosso pipeline simples, grande parte é sequencial (precisamos dos dados para treinar), então a paralelização interna pode estar mais dentro das etapas (por exemplo, treinar múltiplos modelos candidatos em paralelo). Isso nos leva a outro ponto: podemos paralelizar não só as etapas, mas os experimentos. Por exemplo, em vez de rodar serialmente cinco configurações de hiperparâmetros, podemos rodar cinco instâncias do pipeline em paralelo (cada uma com uma configuração) se tivermos recursos computacionais para tal, usando sistemas de distribuição de experimentos (como otimização Bayesiana com múltiplos workers). Em suma, paralelize tudo o que der, respeitando limites de custo e hardware – isso requer às vezes escrever o pipeline de modo thread-safe ou configurável para processos distintos.

Para conjuntos de dados extremamente grandes, pode ser ineficiente processar todo o dataset sempre. Uma estratégia de performance é adotar processamento incremental: em vez de reprocessar todos os dados históricos quando algo muda, processar apenas as adições ou modificações.

Por exemplo, se recebemos dados novos diariamente, podemos projetar o pipeline para ingerir apenas o delta do dia e atualizar o modelo incrementalmente. O

DVC por si só não implementa esse controle de incremental (ele trata arquivos/diretórios como unidades), mas podemos subdividir nossos dados por partições (ex.: por data) e ter cada partição como entrada separada.

Assim, se somente a partição de “10 de fevereiro” é nova, apenas essa entrará no pipeline de preparação. Técnicas de streaming e processamento em blocos também ajudam: por exemplo, usar frameworks que suportam ler dados por chunks (pandas permite ler um CSV em pedaços para não carregar tudo na memória).

Para modelos, existe o conceito de treino contínuo: algoritmos como SGD podem atualizar um modelo existente com dados novos sem precisar recomeçar do zero. Em pipelines avançados, pode-se implementar “treino incremental” – mas isso requer cuidado para não introduzir drift. De toda forma, pensar: “preciso realmente refazer tudo ou posso aproveitar cálculos já feitos?” é crucial quando a escala cresce.

Às vezes, a resposta será usar técnicas e modelos especializados (por exemplo, um modelo online em vez de batch).

Lembre-se: pipelines integrados não se restringem a retreinamento do zero; podemos incorporar lógica incremental dentro deles para performance.

Quando os dados e modelos estão em armazenamento remoto (ex.: na nuvem), o gargalo pode ser a transferência de dados. Para performance, devemos minimizar downloads/uploads desnecessários. O DVC, por exemplo, faz push/pull somente de diffs, mas se a latência de rede for alta para muitos arquivos pequenos, isso pode pesar. Uma solução é usar remotes “cache” locais ou de rede rápida: por exemplo, configurar um cache compartilhado em uma NAS local para a equipe durante desenvolvimento (assim DVC usa a rede local rápida e sincroniza com a nuvem apenas de tempos em tempos).

Outras dicas: comprimir arquivos pode reduzir tempo de transferência (a custos de CPU) – DVC não comprime por padrão, mas podemos inserir compressão no pipeline (zipar outputs) se fizer sentido. Também, para grandes arquivos, o DVC (e ferramentas subjacentes como boto3 para S3) usam transferência multi-parte, que

paraleliza o upload/download de chunks – garantir que isso esteja habilitado otimiza throughput.

Para otimizar, é útil saber onde o pipeline gasta mais tempo ou recursos. Ferramentas de profiling - como o módulo cProfile do Python, ou logs de tempo manuais, ou APMs – Application Performance Monitoring - podem identificar gargalos: por exemplo, descobrir que 80% do tempo de treino é gasto lendo dados do disco, ou que a etapa X consome 90% da memória. Com esses dados, conseguimos aplicar otimizações direcionadas (talvez converter o formato dos dados para um binário mais rápido de ler etc.). O MLflow pode ajudar registrando métricas customizadas de tempo por etapa.

Em pipelines de produção, integrar com monitores como Prometheus ou DataDog fornece métricas de CPU, memória e I/O durante a execução. Assim, se um pipeline que normalmente leva 1h começa a levar 2h, conseguimos investigar qual etapa degradou. Logging estruturado é importante: cada etapa deve emitir logs do tipo “Iniciando preparação... terminada em 3m20s, 100000 registros processados”. Esses logs permitem pós-análise com facilidade. A cultura de observabilidade está entrando no MLOps: monitorar não só os modelos em prod, mas também o próprio pipeline de treinamento. Isso permite detecção de anomalias no processo (ex.: um dado corrompido pode fazer a etapa de preprocess demorar muito por looping inesperado – se monitoramos o tempo, pegamos isso). Em suma, medir é o primeiro passo para otimizar.

Em projetos práticos, otimizar o pipeline pode envolver testar diferentes soluções. Por exemplo: usar um banco de dados em vez de arquivos CSV para armazenar dados intermediários, se as junções estiverem lentas. A decisão de otimização deve considerar custo-benefício: às vezes, simplesmente adicionar mais recursos computacionais (escalar verticalmente ou horizontalmente) pode ser suficiente e mais simples do que reescrever partes do pipeline. Em outras situações, reescrever uma parte em uma linguagem mais rápida (C++ ou usar numba) pode ser válido se for o hotspot. O importante é que nossa arquitetura modular facilita identificar e atacar separadamente os gargalos, sem necessidade de refazer todo o pipeline.

Lembre-se de que você pode rever as partes mais desafiadoras deste conteúdo assistindo às videoaulas para recapitular os conceitos teóricos chave e para

consolidar a prática do pipeline em ação! Os vídeos estão disponíveis para consulta sempre que você precisar revisar esses tópicos.

EMSE

MERCADO, CASES E TENDÊNCIAS

Cases de sucesso e adoção na indústria

A integração de pipelines DVC + MLflow reflete práticas adotadas por líderes de tecnologia. Por exemplo, a Uber desenvolveu a plataforma interna Michelangelo e o Facebook criou o FBLeaRner Flow para gerenciar centenas de modelos em produção – ambas seguem a filosofia de pipelines automatizados com versionamento e experimentação padronizada. Hoje, ferramentas abertas como DVC e MLflow democratizam esse poder: empresas de diversos setores relatam ganhos significativos. A Microsoft e a Databricks estão entre as organizações que adotaram o MLflow extensivamente, contribuindo para sua evolução. A Databricks, por exemplo, incorporou o MLflow em seu produto e observou milhares de usuários ativos na comunidade – o projeto alcançou mais de 20 mil estrelas no GitHub em 2025, demonstrando forte adoção. Um caso concreto: a Airbnb implementou pipelines reproduzíveis para atualizar periodicamente seus modelos de preço dinâmico, utilizando um fluxo semelhante ao que estudamos (embora usando ferramentas próprias); isso reduziu drasticamente erros e tempo de deploy de novos modelos. Relatos indicam ciclos de implantação duas vezes mais rápidos após padronização MLOps.

No mercado financeiro, o JPMorgan divulgou que emprega DVC para cumprir requisitos regulatórios de reprodutibilidade em modelos de risco – cada modelo aprovado pode ser reconstruído bit a bit meses depois graças ao versionamento de dados e código pelo pipeline. Já a Tencent (China) relatou ter integrado MLflow para acompanhar experimentos de NLP com milhares de variações, conseguindo identificar configurações ótimas que antes passariam despercebidas. Esses exemplos ilustram que pipelines integrados não são apenas “boa engenharia”, mas sim um diferencial competitivo: empresas conseguem iterar mais rápido nas soluções de IA e trazê-las ao mercado com confiança.

Tendências atuais em MLOps e pipelines

Várias tendências despontam no cenário de pipelines de ML:

Automação completa de CI/CD de ML: cada vez mais, empresas buscam pipelines de ML totalmente automatizados, onde desde o preparo de dados até o deploy do modelo em produção ocorram sem intervenção humana (após um trigger, como chegada de novos dados ou um agendamento periódico). Ferramentas de CI/CD, como GitHub Actions, GitLab CI ou Jenkins, estão sendo configuradas para acionar dvc repro e testes de ML assim que mudanças são integradas ao Git. Isso cria um processo contínuo: integrações de código de ML passam por testes automatizados e, se aprovadas, já treinam e publicam um novo modelo. MLOps nível 2 (segundo definições da Google) é justamente essa convergência de pipeline de ML + pipeline de deploy de software. É esperado que nos próximos anos isso se torne padrão – semelhante ao que DevOps fez com aplicativos web.

Orquestradores especializados e servidores de pipeline: embora DVC e MLflow operem em nível de código e arquivos, vê-se a ascensão de plataformas orquestradoras visuais de pipelines. Por exemplo, o Kubeflow Pipelines (da Google) permite desenhar um pipeline e executá-lo em um cluster Kubernetes, integrando fácil com CI/CD na nuvem. Da mesma forma, AWS Sagemaker Pipelines e Azure ML Pipelines oferecem serviços gerenciados onde se definem steps (em Python ou com interface gráfica) e o serviço cuida de provisionar recursos, paralelizar, versionar outputs no cloud storage e registrar métricas. Essas plataformas muitas vezes integram DVC/MLflow internamente ou oferecem funcionalidade semelhante (versionamento e experiment tracking nativos). A tendência é pipelines cada vez mais “como código” (infraestrutura-as-code aplicada a ML): o indivíduo engenheiro define o pipeline em YAML ou em código alto-nível e a plataforma executa. Porém, conhecer os fundamentos (como fizemos manualmente com DVC/MLflow) continua crucial para usar bem essas ferramentas.

Governança e compliance mais rígidos: com a proliferação de modelos de ML impactando decisões sensíveis (finanças, saúde, justiça), órgãos reguladores exigem auditabilidade completa. Isso impulsiona adoção de pipelines integrados em setores além de tecnologia – p.ex., bancos implementando Model Governance

incluem requisitos de DVC/MLflow ou equivalentes para garantir que cada modelo aprovado tenha histórico de treinamento verificável. A UE, por exemplo, discute regulações de AI que demandariam documentação de dados e experimentos de qualquer modelo de alto risco. Assim, pipelines integrados deixam de ser apenas preferíveis e tornam-se necessários por exigência legal em alguns contextos.

Pipelines como código aberto consolidado: ferramentas open-source como DVC, MLflow, Airflow, Metaflow (Netflix) e outras estão se consolidando como stack padrão de MLOps em muitas empresas, substituindo soluções caseiras. A interoperabilidade entre elas também melhora – por exemplo, já existem plugins para sincronizar MLflow Runs com DVC Experiments, unificando tracking. A comunidade caminha para padrões abertos: o ML Metadata (MLMD) do TensorFlow e o OpenLineage (projeto LFAI) tentam padronizar o formato de metadata de pipelines. Isso significa que no futuro diferentes ferramentas podem compartilhar metadata – imagine treinar com DVC/MLflow e visualizar lineage e metrics em outra interface, graças a padrões comuns.

Leituras e recursos recomendados:

[Artigo](#): “Machine Learning: The High-Interest Credit Card of Technical Debt” – Sculley et al., NIPS 2015. Este é o famoso paper do Google que motivou grande parte do movimento MLOps, discutindo exatamente a necessidade de arquiteturas de pipeline melhores.

[Whitepaper](#): “pipelines de automatização e entrega contínua na aprendizagem automática”: Discute níveis de maturidade em MLOps e ilustra pipelines de referência de ponta a ponta.

Livro: “Introducing MLOps” – Mark Treveil et al., O’Reilly (2020): aborda casos de uso reais e melhores práticas para implementar MLOps em empresas, com capítulos sobre versionamento de dados e experimentos.

[Vídeo](#) (YouTube): “MLOps Tutorial: Build a Full ML Pipeline with MLflow, DVC & Deploy on AWS” – Analytics Vidhya (2025) – um passo a passo prático demonstrando a criação de um pipeline integrado usando exatamente DVC e MLflow,

culminando no deploy em AWS (ótimo para ver a aplicação do que aprendemos em contexto real).

Podcast: “Machine Learning Street Talk – Episode: Reproducibility in ML” (2023) – Especialistas da Netflix e Microsoft discutem desafios e soluções para reprodutibilidade; compartilham experiências sobre implementação de pipelines com meta-dados completos.

O ecossistema está evoluindo rapidamente. De olho no futuro, vemos a emergência de AutoML pipelines onde grande parte do fluxo (incluindo geração de features e seleção de modelos) é automatizada, mas ainda assim sob o guarda-chuva do MLOps para garantir repetibilidade. Também há avanço em pipelines federados – quando dados estão distribuídos (por privacidade), o pipeline orquestra treinamento em vários nós sem centralizar os dados. Esses cenários trazem considerações adicionais (como versionamento de modelos agregados etc.) que provavelmente gerarão extensões das ferramentas atuais.

Em resumo, o mercado caminha para tornar pipelines de ML mais robustos, padronizados e integrados ao restante da TI. Para o(a) profissional de dados, dominar essas ferramentas e conceitos não é mais diferencial, mas sim requisito básico para escalar soluções de IA no mundo real.

Referências de mercado & tendências

“MLOps Market Growth 2025” – [Arcade.dev Blog](#): análise de tendências de adoção de MLOps e investimento. Mercado global projetado em \$39 bi até 2034, 87% das grandes empresas usando IA.

“Reproducible machine learning with DVC + MLflow” – SURF Research (2021): [guia prático](#) mostrando uso combinado de DVC e MLflow em um case de pesquisa (disponível no portal SURF).

“Data Versioning in MLOps: Best Practices for Reproducible Pipelines” – CodezUp (2025): [artigo](#) tutorial detalhando implementações de versionamento de dados com DVC, MLflow e BentoML.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, compreendemos como integrar DVC e MLflow em pipelines de Machine Learning, criando fluxos automatizados, reproduzíveis e confiáveis para projetos de modelos. Vimos, passo a passo, os princípios para estruturar um pipeline com etapas bem definidas (preparação dos dados, treinamento do modelo, avaliação) e conectadas via orquestração declarativa. Aprendemos a versionar dados e modelos de forma semelhante ao versionamento de código, usando DVC para garantir reprodutibilidade total dos artefatos, e a rastrear experimentos com MLflow, registrando parâmetros, métricas e resultados de cada tentativa de modelo. Exploramos conceitos de MLOps que amarram esses componentes – da automação da execução condicional (reexecutar somente o necessário) até a importância de testar e monitorar nossos pipelines continuamente.

Você também viu exemplos práticos de código integrando as ferramentas (como snippets de `dvc.yaml` e `train.py` com MLflow) e entendeu como esses elementos se encaixam em um fluxo coeso. Discutimos ainda boas práticas de engenharia aplicadas: separar claramente responsabilidades em cada etapa, priorizar reuso e paralelismo, gerenciar ambientes de execução e proteger a qualidade do pipeline através de testes e validações. Ao final, conectamos esses aprendizados ao estado atual da indústria, observando que as técnicas dominadas aqui – versionamento de dados, tracking de experimentos, pipelines automatizados – são altamente valorizadas e adotadas pelas principais empresas de tecnologia dentro de seus processos de IA.

Parabéns por avançar nessa jornada! Agora você dispõe de habilidades práticas de MLOps que fazem a ponte entre a teoria dos modelos e a entrega de valor no mundo real.

REFERÊNCIAS

ANALYTICS VIDHYA. **MLOps Tutorial: Build a Full ML Pipeline with MLflow, DVC & Deploy on AWS.** YouTube (vídeo), 2025. Disponível em: https://www.youtube.com/watch?v=oYIBwbHM_PI. Acesso em: 24 mar. 2026.

ARCADE. **Dev Team. MLOps Market Growth 2025: \$39B Expansion, 87% Enterprise Adoption & AI Scaling.** Blog Arcade.dev. 2025. Disponível em: <https://blog.arcade.dev/mlops-community-expansion-trends>. Acesso em: 24 mar. 2026.

BERBERI, L.; KOZLOV, V.; NGUYEN, G. et al. **Machine learning operations landscape: platforms and tools.** Artificial Intelligence Review, v.58, art.167, 2025. DOI: 10.1007/s10462-025-11164-3.

CODEZUP. **Data Versioning in MLOps: Best Practices for Reproducible Pipelines.** Codezup.com (artigo online), 06 maio 2025. Disponível em: <https://codezup.com/data-versioning-in-mlops-best-practices/>. Acesso em: 24 mar. 2026.

DATA VERSION CONTROL (DVC). **Documentação do DVC.** Iterative Inc., 2025. Disponível em: <https://dvc.org/doc>. Acesso em: 24 mar. 2026.

GARG, S. et al. **On Continuous Integration / Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps.** arXiv preprint arXiv:2202.03541, 2022. Disponível em: <https://arxiv.org/abs/2202.03541>. Acesso em: 24 mar. 2026.

GOOGLE CLOUD. **MLOps: Continuous delivery and automation pipelines in machine learning.** Documentação do Cloud Architecture Center, 2021. Disponível em: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning?hl=pt>. Acesso em: 24 mar. 2026.

MLFLOW. **MLflow Documentation.** Databricks, 2025. Disponível em: <https://mlflow.org>. Acesso em: 24 mar. 2026.

SCULLEY, D. et al. **Hidden technical debt in machine learning systems. In: Advances in Neural Information Processing Systems (NeurIPS),** v.28, p.2503–2511, 2015. Disponível em: <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>. Acesso em: 24 mar. 2026.

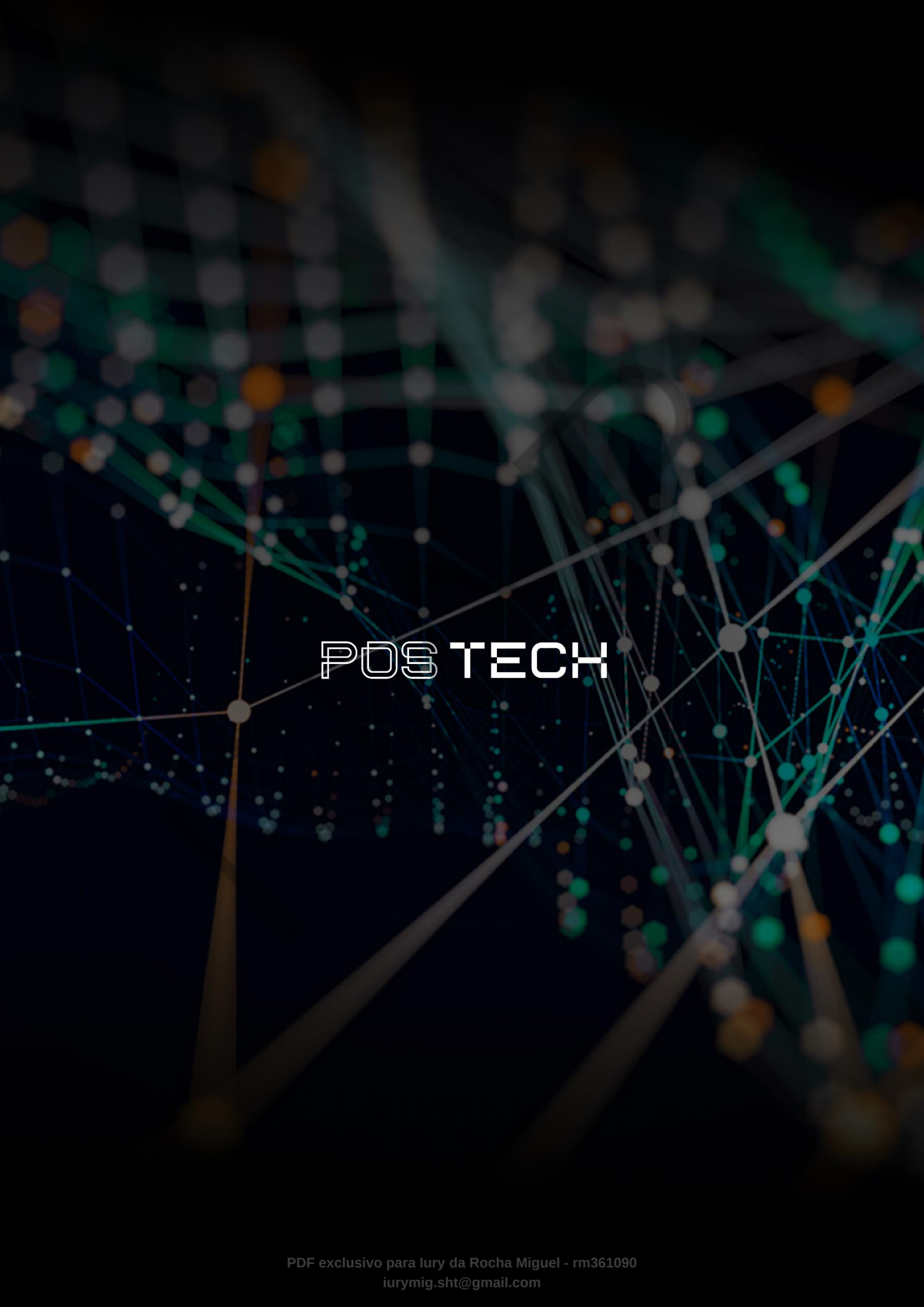
SIG **MLOps. MLOps Principles.** 2023. Disponível em: <https://ml-ops.org/content/mlops-principles>. Acesso em: 24 mar. 2026.

ZAHARIA, M. et al. **Accelerating the Machine Learning Lifecycle with MLflow.** IEEE Data Engineering Bulletin, v.41, n.4, p.39–45, 2018.

PALAVRAS-CHAVE

Automação. MLOps. Pipelines de Machine Learning.

EMEND



POSTECH